

ПРОЕКТ 1. ОСНОВНОЙ КОМАНДНЫЙ WEB-ПРОЕКТ

Название

Децентрализованная система для колледжа ITSTEP

Формат

Команда из 2 человек

Цель

Разработать web-приложение, объединяющее функции студенческой соцсети, системы ориентации внутри колледжа и внутреннего цифрового помощника.

5.1. ОБЯЗАТЕЛЬНЫЕ ФУНКЦИИ ПРОЕКТА 1

1. Аутентификация и авторизация

Нужно реализовать:

- регистрацию;
- вход;
- выход;
- хранение сессии / токена;
- разграничение ролей.

2. Профиль пользователя

В профиле должны быть:

- имя;
- фамилия;
- группа / курс;
- направление;
- аватар;
- краткая информация о себе;
- список клубов / интересов;
- друзья;
- посты пользователя.

3. Лента постов

Пользователь должен уметь:

- создавать пост;
- редактировать свой пост;
- удалять свой пост;
- просматривать посты других;
- ставить реакции;
- писать комментарии.

4. Друзья

Необходимо реализовать:

- отправку заявки в друзья;
- принятие / отклонение заявки;
- удаление из друзей;
- список друзей.

5. Чат / общение

Минимум:

- личные сообщения между пользователями

Желательно:

- групповые чаты
- чат клуба / сообщества

6. Клубы и кружки

Должны быть:

- карточки клубов;
- описание;
- расписание / время встреч;
- контакты ответственных;
- возможность подписаться / вступить.

7. Новости колледжа

Должны быть:

- новости;
- объявления;
- обновления мероприятий;
- дедлайны / события.

8. Навигация для студентов

Нужно реализовать раздел, где студент может понять:

- какие кружки есть;
- куда обращаться по разным вопросам;
- где искать расписание;
- как найти преподавателя / администратора;
- какие активности доступны.

9. Поиск

Минимум поиск по:

- пользователям;
- постам;
- клубам;
- новостям.

10. Админ-функции

Минимум:

- управление новостями;
- модерация постов;
- управление клубами;
- управление пользователями.

5.2. ОБЯЗАТЕЛЬНЫЕ НЕФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ ПРОЕКТА 1

Архитектура

Архитектура должна быть понятной, чистой и логично разделенной.

Для фронтенда ожидается:

- разделение на pages / features / components / shared либо другой внятный подход;
- отсутствие хаоса в папках;
- вынос API-запросов;
- разделение UI и логики;
- переиспользуемые компоненты.

Для бэкенда ожидается:

- router;
- handlers/controllers;
- services/usecases;
- repositories;
- models/entities;
- config;
- middleware;
- utils.

Нельзя сдавать проект в стиле “весь код в одном файле”.

Git

Обязательно:

- работа через GitHub;
- минимум 1 репозиторий на команду;
- у каждого студента должны быть свои коммиты;
- понятные commit messages;
- работа через ветки приветствуется.

API

Обязательно:

- REST API;
- корректные HTTP методы;
- адекватные статусы ответа;
- JSON-ответы;
- документация через Postman.

Документация

Обязательно приложить:

- README;
- инструкцию по запуску;
- архитектурную схему;
- ERD;
- описание SQL и NoSQL частей.

ЧТО ИМЕННО ДОЛЖНЫ ОБЪЯСНЯТЬ СТУДЕНТЫ НА ЗАЩИТЕ

По аналитике

Студенты должны объяснить:

- какую проблему решает проект;
- кому он нужен;
- почему этот продукт не является копией обычной соцсети;
- чем он полезен именно для колледжа;
- кто целевая аудитория;
- как они проводили анализ аудитории;
- кто конкуренты и в чем отличие.

По UX/UI

Студенты должны объяснить:

- почему интерфейс выглядит именно так;
- как пользователь идет по user flow;
- как выглядит путь нового студента;
- как через продукт решается проблема адаптации и коммуникации.

По frontend

Студенты должны объяснить:

- как устроена архитектура;
- почему выбран именно такой state management;
- зачем нужны кастомные хуки;
- почему выбрана конкретная CSS/UI библиотека;
- как устроена маршрутизация;
- как фронт работает с API.

По backend

Студенты должны объяснить:

- как устроен GIN-проект;
- где роуты, где хендлеры, где сервисы, где репозитории;
- как работают middleware;
- как обрабатываются ошибки;
- как реализованы эндпоинты;
- как бэк связан с фронтом и БД.

По БД

Студенты должны объяснить:

- какие данные лежат в SQL;
- какие в NoSQL;
- почему именно так;
- как построены связи;
- как работает ERD;
- как обеспечивается целостность данных.

По командной работе

Студенты должны объяснить:

- как они делили задачи;
- кто за что отвечал;
- как работали в Git;
- как решали конфликты;
- как собирали проект вместе.